Brief Introduction

Discover why it's a great time to master Kotlin and learn this programming language, who this course is for, and what you'll learn in this course.

WE'LL COVER THE FOLLOWING

- What is Kotlin?
  - What Can I Use Kotlin For?
- Who's Using Kotlin?
- Why Learn Kotlin?
- What This Course Covers
- Summary

What is Kotlin?

Kotlin is an open-source, statically typed programming language. It can transpile to Java bytecode to run on the Java Virtual Machine (JVM) or Android. Interestingly, it can also transpile to JavaScript code or native bytecode, allowing you to use it for virtually anything from web applications to iOS apps to embedded devices and more. This course focuses on Kotlin for the JVM.

> **Note:** The term "transpile" refers to translating a piece of code between languages or representations. For instance, transpiling Kotlin code to Java bytecode means that a piece of code in Kotlin syntax is translated to its corresponding representation in Java bytecode. Thus, the syntax changes but the semantics remains the same.

Kotlin is developed by JetBrains, the company behind the IntelliJ IDE and other development tools. **They designed Kotlin to be 100% interoperable with Java, meaning that you can use any Java libraries and frameworks in**

Kotlin, and vice versa. Keep this in mind, as we will make use of this interoperability throughout the course.

What Can I Use Kotlin For?

As previously mentioned, **you can use Kotlin on any platform**. Currently, it's most prominent for Android app development. However, its usage for developing server-side applications for the JVM is increasing.

Additionally, it can be used to write web applications by transpiling to JavaScript. Kotlin then lets you use popular libraries and frameworks such as React and VueJS.

It's useful to know about the corresponding sub-projects in case you encounter them:

- **Kotlin/JVM:** lets you write JVM applications by transpiling to Java bytecode (e.g., Android development, Spring services, desktop applications)
- **Kotlin/JS:** lets you write web applications by transpiling to JavaScript (e.g., React apps, and Vanilla JS apps)
- **Kotlin/Native:** lets you write applications for any target platform such as embedded devices or iOS (e.g., iOS apps, microcontroller programming)

> As a side note, Kotlin supports multi-platform application development, which enables you to share code between apps for different platforms. The most obvious use case is to share core functionality between your Kotlin Android app and your Kotlin iOS app.

Who's Using Kotlin?

By now, Kotlin is used by too many large companies to name them all. But just to convince you that Kotlin is used in production by some of the largest tech companies in the world, here's a selection of them:

- Google
- Netflix

- Uber
- Slack
- Udacity
- Evernote
- Trello
- Slack
- Lyft
- ...

Many of these companies use Kotlin on Android, but also on server side and other platforms.

Why Learn Kotlin?

Personally, learning Kotlin has taught me a lot about language design and some of the shortcomings of Java that I wasn't aware of previously. This was back in 2015 before Kotlin version 1.0 was out. Therefore, it can be a great way to learn about modern programming language features that you will encounter in other languages as well, such as TypeScript, Scala, Go or Rust.

Apart from this, there are many **companies actively looking for Kotlin developers, especially in the Android development space**. So having experience in Kotlin can give you an edge in the interview process.

But even without the goal of landing a Kotlin developer job, learning the language and its concepts will allow you to **quickly master other modern languages such as TypeScript, Scala, and Swift** because they share many language concepts.

What This Course Covers

This course covers the basics of the Kotlin language:

- Variables and data types, including type inference and one of my favorite Kotlin features: nullable types
- Conditions using `if` and `when`, why `when` is more powerful than `switch` in other languages, and how to use conditions as expressions
- Collection types from lists to sets to maps (and arrays), with notes about

the importance of immutability

- Loops for iterative control flow and how Kotlin's `for` differs from `for` loops you may be familiar with
- Functions, including Kotlin's shorthand notation for simple functions, how to use default values and named parameters, Kotlin's extension functions, and how to use infix functions and custom operators to improve your code's readability

Each lesson ends with a quiz to review the main takeaways and includes at least one hands-on programming exercise. Doing the exercises is the most important part of the course since these will give you the most value in terms of mastering Kotlin.

Summary

Kotlin is a modern language that made its way into the mainstream over the last few years, with Google I/O 2017 being one major accelerator of this development.

- It can run on any platform by transpiling to Java bytecode, JavaScript or native bytecode
- Large companies use Kotlin and continue to have a positive experience with it, proving again that it's ready for adoption at larger scale
- Learning Kotlin can land you a job and helps you master other modern languages faster
- This course is the entry point into your Kotlin journey, teaching you all you need to write your first simple applications

---

In the next lesson, you will learn about Kotlin's principles and goals and how they guide the design of the langauge.

Kotlin Overview: Principles and Goals

Find out Kotlin's guiding principles, how they affect language design, and why this is relevant for developers.

WE'LL COVER THE FOLLOWING

- Principles and Goals
  - Implications for Developers
- Summary

Kotlin's language design is guided by its underlying principles and goals.

Principles and Goals

Kotlin aims to be a readable, pragmatic, safe, and interoperable programming language:

- **Readability** is supported by language features such as type inference, data classes, and infix functions. Such features allow writing concise code without losing readability.
- **Pragmatism** is crucial because Kotlin is for large-scale enterprise software development. JetBrains use it themselves to develop their IDEs. Thus, Kotlin incorporates industry feedback and addresses issues of large-scale software development.
- **Safety** aims to prevent common software bugs by design. This is aided by several language features such as nullable types (to prevent null pointer exceptions) and by nudging you towards best practices such as designing for inheritance.
- **Interoperability** with Java is a major selling point of Kotlin and a necessary base for its widespread adoption in the JVM world. Interoperability allows Kotlin and Java to be used side by side, including the use of Java libraries or frameworks from Kotlin. For instance, the Kotlin standard library interoperates with Java by reusing the Java

Collections API. Similarly, it interoperates with JavaScript in the context of Kotlin/JS.

> **Note:** The concept of interoperability with Java will be used in the code examples in this course. For instance, we will use types from the Java standard library, such as `Date`.

Implications for Developers

Kotlin's principles guide its language design decisions and, as a result, the way you develop in Kotlin.

- Kotlin makes you think about nullability from the start by integrating nullability into the type system.
- Kotlin is intentionally constrained in certain areas (e.g. operators) to aid readability.
- Kotlin favors immutability and functional programming, for instance by nudging you to use read-only (`final`) variables.
- Kotlin drops checked exceptions to aid scalability in large-scale software development.
- Since it was developed by a tooling company, Kotlin focuses on toolability. For example, variable types are defined after variable names to remove ambiguity while parsing for the compiler (but also due to type inference and other reasons).

Summary

In short, it's important to know the philosophy of a language to understand why some things work the way they do. Ideally, your personal development philosophy matches with that of the language you use.

Next up is a brief lesson with some tips on how you can make the most of this course.

Following this Course

Learn how to make the most of this course on Kotlin.

WE'LL COVER THE FOLLOWING

- Before you start your Kotlin journey...
- Maximizing your Learning

Before you start your Kotlin journey...

A few key notes about this course:

- The code examples shown in this course may have **hidden code**.
  - This is used at the beginning of the course to facilitate writing your first lines of Kotlin code.
  - This is useful to print outputs when you run a code example.

Maximizing your Learning

- **Please complete all the quizzes and exercises.**
- You can find all exercise **solutions on GitHub**.
  - **Solve them yourself first!** Spend time on these if necessary. It's the most valuable part of the course and key to your learning!

Now let's dive into the hands-on part of the course!

Read-Only vs Mutable Variables

Learn the fundamental principle of read-only and mutable variables, plus how and when to use them in Kotlin.

WE'LL COVER THE FOLLOWING

- Mutable Variables
- Read-Only Variables
- Quiz
- Exercise
- Summary

Let's start with the very basics: declaring variables. Kotlin fundamentally differentiates between read-only and mutable data.

Mutable Variables

To declare a mutable variable, you use the `var` keyword:

```
var number = 17
println("number = $number")

number = 42  // var can be reassigned
println("number = $number")
```

Mutable means that the variable can be reassigned to a different value after initial assignment.

Read-Only Variables

In contrast, a read-only variable can be declared using `val` (instead of `var`):

```
val number = 17
println("number = $number")



number = 42  // Not allowed, throws an exception
```

Read-only means that the variable cannot be reassigned once initialized.

You should prefer read-only variables to mutable ones whenever possible, i.e., whenever you don't have to change the value after initialization.

> **Tip**: Prefer `val` to `var` to simplify data flow and facilitate reasoning about your code.

Quiz

Readonly and Mutable Variables

1 Which of the following code snippets correctly initialize readonly variables? You can select multiple answers.

Exercise

Fill in the gaps in this code to create a mutable variable `age` and a read-only variable `name`:

Problem | Solution

```
// Create your variables
```

Summary

Kotlin differentiates between read-only and mutable variables at declaration time, forcing you to think about the mutability of state from the start.

- `val` lets you declare read-only variables.
- `var` lets you declare mutable variables.
- Good practice is to prefer `val` over `var` whenever possible to reduce mutability and therefore facilitate understanding of the program's state and data flow.

In the next lesson, you will learn about Kotlin's fundamental data types and how to use them.

Basic Data Types

Learn how to represent and store numbers and text in Kotlin.

WE'LL COVER THE FOLLOWING

- Integers
- Floating Point Numbers
- Text
- Booleans
  - Note: Type Mapping to Java
- Quiz
- Exercise
- Summary

Kotlin is a statically typed language, meaning that the data type of every expression is known at compile time.

Integers

There are four basic data types to store integer numbers of different sizes in Kotlin:

```
val byte: Byte = 127
val short: Short = 32767
val int: Int = 2147483647
val long: Long = 9223372036854775807
```

> **Note:** The values assigned to each variable above are the largest allowed for the corresponding data type. This gives you an indication of their

magnitude.

If you increase the assigned values even by one, the Kotlin compiler will complain because an overflow would occur at runtime (*try it!*).

Floating Point Numbers

Additionally, Kotlin has `Float` and `Double` to store floating point numbers up to different precision and sizes:

```
val float: Float = 3.4028235e38f
val double: Double = 1.7976931348623157e308
```

Here again, the assigned values are the maximum allowed for the corresponding type.

Two things to note:

- The `e` in both values denotes exponentiation, for instance `10e3 == 1000`.
- In order to denote a `Float` value, you have to add the `f` suffix. Otherwise, Kotlin infers `Double` as the type of the number.

Text

Kotlin uses the `Char` type for single characters and `String` for arbitrary sequences of characters:

```
val character: Char = '#'
val text: String = "Learning about Kotlin's data types"
```

Single characters are denoted using single quotes `''`, whereas basic strings use double quotes `""`.

However, you can also use multiline strings by wrapping your string into three double quotes: `"""<multiline string here>"""`.

Booleans

Finally, Kotlin uses `Boolean` to store either `true` or `false`:

```
val yes: Boolean = true
val no: Boolean = false
```

These two are the only valid values for the `Boolean` type.

Note: Type Mapping to Java

Since Kotlin transpiles to Java bytecode (if you're using it on the JVM), it's important to explore how language concepts translate to Java.

In contrast to Java, Kotlin has no primitive types. All types discussed in this lesson are objects at runtime. However, they do transpile to Java's primitive types in Java bytecode:

Kotlin Type	Type in Java Bytecode
`Byte`	`byte`
`Short`	`short`
`Int`	`int`
`Long`	`long`
`Float`	`float`
`Double`	`double`
`Char`	`char`
`Boolean`	`boolean`

The mapping is quite trivial, but note that this mapping only works because Kotlin's types cannot be null by default, just like Java's primitive types cannot hold `null`. Kotlin's approach to null safety is explained in detail later in the course.

Quiz

Data Types in Kotlin

1 Which of these are valid data types to hold *integers* in Kotlin?

COMPLETED 0%

1 of 3

Exercise

Create a variable `favoriteMovie` that stores the title of your favorite movie and a variable `rating` that contains your rating for it. Ratings range from 0.5 to 5.0 in increments of 0.5.

Problem Solution

```
// Add your variables below
```

Summary

Here's what you should take away from this lesson:

- Kotlin has `Byte`, `Short`, `Int`, and `Long` as basic types for integer numbers.
- Kotlin uses `Float` and `Double` for floating point numbers. A `Float` is denoted with a trailing `f`, as in `17f`.
- Kotlin has `Char` to store single characters and `String` to store strings of text.
- Kotlin's basic types map to Java's primitive types when targeting the JVM (and `String` maps to `String`).

In the following lesson, you'll understand how and when the Kotlin compiler can infer the types used in your code.

Kotlin's Type Inference

Understand Kotlin's powerful type inference capabilities and how to use them to write more succinct code.

WE'LL COVER THE FOLLOWING

- Type Inference in Kotlin
  - Type Inference for Literals
  - Type Inference for Objects
  - More Type Inference
- Quiz
- Summary

Type inference is a compiler feature that allows you to omit types in your code when the compiler can infer it for you.

Type Inference in Kotlin

Kotlin’s compiler can infer the types of most variables, so adding the type is optional:

```
// Run the code to see the variable's types
val string = "Educative"
val int = 27
val long = 42L
val double = 2.71828
val float = 1.23f
val bool = true
```

> **Note:** The terms on the left-hand side of the equals sign are just the variable names, not the data types.

Type Inference for Literals

The compiler automatically infers that the `string` variable must have type `String` because it's assigned to the value `"Educative"`.

For integer values, the compiler infers `Int` by default. You can use the `L` suffix as in `42L` to transform the value into a `Long`. Similarly, the compiler infers `Double` for floating point numbers unless you add an `f` as suffix as in `1.23f`, in which case it is considered a `Float`. There's no such shortcut suffix for `Byte` or `Short` because these aren't used as often.

> If you run the code above, it will print the mapped Java type for each variable at runtime.
>
> (Kotlin reflection is currently not available on Educative so it prints the Java types. These are evaluated at runtime using `string.javaClass`, `int.javaClass`, and so forth.)

Type Inference for Objects

Naturally, type inference doesn't only work with literal values on the right-hand side. The compiler can infer types of object just as easily:

```
import java.util.*
import java.time.*
import java.io.*

val stringBuffer = StringBuffer("PREFIX")
val localDate = LocalDate.now()
val file = File("foo.txt")
```

Essentially, as long as it's possible to infer the variable type, Kotlin's compiler can do so.

> **Tip:** You may still choose to *add explicit types for clarity* or to use a *more abstract type* than what would be inferred. It is good practice to program against abstract interfaces where it makes sense.

More Type Inference

Apart from simple variable types, Kotlin's compiler can infer types in lambda expressions, function return types, generic type parameters, and so forth – given sufficient context.

Using type inference is fundamental in idiomatic Kotlin code so you won't see many explicit types in this course. If you're working in a larger Kotlin codebase with IntelliJ, you can switch on type hints to let the IDE show you the inferred types. This may be helpful while getting used to inferred types.

Quiz

To understand the compiler... you must become the compiler.

1 Which type is inferred for "17"?

COMPLETED 0%

1 of 3

Summary

Here are the key takeaways from this lesson:

- The Kotlin compiler can infer most types.
  - Writing down the type explicitly is optional in these cases.
- You may prefer to use explicit types for clarity.
  - Especially when the right-hand side is a complex expression or

function call.

- Inferred types are ubiquitous in idiomatic Kotlin code.

In the next lesson, you will discover Kotlin's fundamental feature of nullable types which make every use of `null` in your code explicit and well-defined.

Nullable Types

After this lesson, you'll be able to recognize and use nullable types, and know how to work with nullables effectively by using Kotlin's special operators to safely access potentially null data.

WE'LL COVER THE FOLLOWING

- Declaring Nullable Types
- Using Nullable Objects
  - Safe Call Operator
  - Elvis Operator
  - Unsafe Call Operator
- Effective Nullability Handling
- Quiz
- Exercise
- Summary

Kotlin's type system differentiates between nullable and non-nullable types. By default, types are non-nullable; you cannot get a null pointer exception from dereferencing it.

This is why Kotlin's basic data types, such as `Int`, can map safely to Java's primitive types, such as `int`, in the bytecode. Both can never be `null`.

Thus, all objects and types you've seen so far in this course were non-nullable. Trying to assign `null` to them would cause a compile-time error:

```
// Compile-time error: cannot assign null to non-nullable type
val input: String = null
```

Declaring Nullable Types

Declaring Nullable Types

Let's see how you can use nullable types when necessary.

The nullable counterpart of some type `A` is denoted as `A?`. For example, `String?` denotes a nullable string and `Int?` denotes a nullable integer.

> If you're familiar with union types (e.g. from TypeScript), think of these as `String? = String | null`. In other words, a `String?` is either a `String` or `null`.

Assigning `null` to a nullable type works as you'd expect:

```
val input: String? = null
```

Using Nullable Objects

Kotlin is focused on safety. Therefore, you cannot simply access any members on a nullable object as this might cause a null pointer exception:

```
val input: String? = null

val output = input.toUpperCase()  // Compile-time error: unsafe call on nullable object
```

Here, the compiler complains because `input.toUpperCase()` will fail at runtime if `input` is `null`, which would indeed be the case here.

Safe Call Operator

To access members of a nullable object in Kotlin, you use the *safe call operator* by appending a `?` to the object:

```
val input: String? = null

val output = input?.toUpperCase()  // Works and returns null
```

The safe call operator works as follows:

- If the object is `null`, it evaluates to `null`.
- Otherwise, it dereferences the objects and evaluates to the value of the overall expression.

> **Exercise:** Try assigning an actual string to the `input` variable above and run the code again.*

In nested structures of nullable objects, you may chain the safe call operator as in `person?.spouse?.name`.

Elvis Operator

When working with nullable data, you often want to define default values that should be used in case an object is `null`.

In Kotlin, this is done using `?:`, the so-called *Elvis operator*:

```
val name: String? = null

val chatName = name ?: "Anonymous"  // Elvis operator
val displayName = chatName.toUpperCase()
```

You can read the Elvis operator as "or else". Here, `chatName` is equal to `name` *or else* `"Anonymous"`.

This operator is useful to get rid of nullability in your code. It's good practice to replace `null` values with useful default values as early as possible, as this will simplify calling code.

> **Fun fact:** The name "Elvis operator" stems from the fact that it looks like

> an elvis emoji `?:-O`

It's worth mentioning that this operator is not just for setting default values. *Anything you put on its right-hand side is executed if the accessed value is* `null`, so another typical case is throwing an exception:

```
val input: String? = null

val userInput = input ?: throw IllegalArgumentException("Input must not be null.")
```

Unsafe Call Operator

The last way to access a member on a nullable object (and literally the last one you should typically use) is the *unsafe call operator* `!!`.

This operator represents a *non-null assertion*. With it, you're telling the compiler that the object you're trying to access cannot be `null` at this particular place in your code. Therefore, Kotlin will access the value without further safety checks. So if you're wrong, you get a null pointer exception.

```
val input: String? = null

val output = input!!.toUpperCase()
```

Using this operator is one of the few ways to get a null pointer exception in Kotlin. *Think of it as an assertion and only use it if necessary*. Usually, you can avoid this by refactoring your code to avoid nullability in the first place or by using the safe operators, as explained above.

Effective Nullability Handling

Explicit nullability, is in my opinion, the primary differentiating feature of Kotlin's type system when compared to Java.

Apart from the obvious benefits regarding safety and clarity, it does have a learning curve. A typical beginner's mistake is to use nullables and `!!`

throughout the code.

Of course, it takes more effort to always think about the “null case”, but Kotlin facilitates this with:

- Operators such as `?` and `?:`
- Various standard library functions such as `String?.isNullOrEmpty()`

Quiz

Nullable types and null handling in Kotlin.

1

Given a nullable `File?` variable called `configFile`, which of the following is a safe way to get the file path?

COMPLETED 0%

1 of 3

Exercise

In the following, you’re given variables `x`, `y`, `z` of type `Double?` (in hidden code that is prepended to your code). Calculate and print the sum `x + y + z`. If a number is `null`, it should be ignored for the sum.

Hint: you can print to the console using `println`.

Problem Solution

```
// Compute sum of x, y, z. Each one may be null (randomized).
```

Summary

Here are the key takeaways from this lesson:

- Kotlin's type system uses explicit nullable types.
  - By default, types are non-nullable (e.g. `Person`).
  - Each type has a nullable counterpart (e.g. `Person?`).
- The safe call operator `?` is used to safely access members of nullable types.
- The Elvis operator `?:` is used to handle null values conveniently (e.g. to define useful default values).
- The unsafe call operator `!!` acts like a non-null assertion and should be used judiciously.

Congratulations! You're now familiar with the basics of Kotlin's type system. In the following section, you'll learn how to implement conditional control flow using `if` and `when`.

Conditions with `if`

Learn how to use Kotlin's `if` conditions as well as comparison and logical operators.

WE'LL COVER THE FOLLOWING

- Conditions using `if`
  - Equality and Comparison Operators
  - Logical Operators
- Quiz
- Exercise
- Summary

For conditional control flow, Kotlin uses the `if` and `when` keywords. The syntax of `if`-blocks is the same as in C and Java. Similarly, the `when` keyword is similar to `switch` in these languages but more powerful, as you will learn.

Conditions using `if`

Conditions with `if` can be written as follows:

```
if (planet == "Jupiter") {
  println("Radius of Jupiter is 69,911km")
} else if (planet == "Saturn") {
  println("Radius of Saturn is 58,232km")
} else {
  println("No data for planet $planet")
}
```

Each condition is followed by a block of code in curly braces. There can be any number of else-if blocks (including zero) and up to one else-block.

> **Note:** In the else-block, the planet variable is included in the output string using *string interpolation*. To do so, you just prepend the variable with a `$`. More complex expressions must be wrapped with curly braces: `"${user.name} logged in"`.

Equality and Comparison Operators

Kotlin's operators to formulate conditions are the following:

Operator	*Meaning*
`==`	`Structural equality ("equals")`
`!=`	`Structural inequality ("!equals")`
`===`	`Referential equality (same memory address)`
`!==`	`Referential inequality`
`>`	`Greater than`
`<`	`Less than`
`>=`	`Greater or equal than`
`<=`	`Less than or equal`

Note that the comparison operators work on any `Comparable` by using its `compareTo` method.

> **Note for Java developers:** In Kotlin, you use `==` where you would use `equals` in Java and `===` where you would use Java's `==`.

Logical Operators

To build up more complex conditions from primitive conditions (using the operators above), Kotlin provides the standard logical operators:

Operator	*Meaning*
`&&`	Logical “and”
`||`	Logical “or”
`!`	Logical “not”

Using these, you can combine multiple primitive conditions:

```
if (planet == "Jupiter" || planet == "Saturn" || planet == "Uranus" || planet == "Neptune") {
  println("Your chosen planet has rings, typically made of rocks and ice boulders")
}
```

Quiz

Conditional control flow using `if`

Q

How would you express the condition “the text is empty or at least 3 characters long”?

COMPLETED 0%

1 of 1

Exercise

Complete the following code snippet to check whether...

1. the `age` variable is between 18 and 21 (both inclusive).
2. the `username` variable equals `"admin"` or `"system"`
3. the `number` variable is equal to neither `17` nor `42`.

Problem | Solution

```
// Note: In this code widget, you have access to the predefined variables age, username, and

// Check age

// Check username

// Check number
```

Summary

- Kotlin's `if` conditions work the exact same way as in other languages like Java or C.
- The logical and comparison operators are also known from some other languages.
  - But in contrast to Java, Kotlin offers `==` for structural equality and `===` for referential equality checks.

In the next lesson, you will use the `when` statement for conditional control

flow and learn when to prefer it over `if`.

Conditions with `when`

Understand Kotlin's `when` construct and best practices of using `if` vs `when`.

WE'LL COVER THE FOLLOWING

- Conditions Using `when`
- Good Practice: `if` vs `when`
- Quiz
- Exercises
- Summary

Conditions Using `when`

You can use `when` conditions to define different behaviors for a given set of distinct values. This is typically more concise than writing cascades of `if`-`else if` conditions. For instance, you can replace the `if` condition above with a `when` condition to save around 30% of the lines of code in this example:

```
when (planet) {
  "Jupiter" -> println("Radius of Jupiter is 69,911km")
  "Saturn"  -> println("Radius of Saturn is 58,232km")
  else      -> println("No data for planet $planet")
}
```

The `when` keyword is followed by the variable it compares against. Then, each line defines the value to check on the left-hand side, followed by an arrow `->` and the block of code to execute if the value matches.

The code block on the right-hand side must be wrapped in curly braces unless it's a single expression. Optionally, an `else`-block can be used to define a block of code to run if no other case matches.

This language construct is basically the same as `switch` from languages like C or Java. However, it's more powerful since it supports more complex checks:

```
when(temperatureInKelvin) {
      700              -> println("This is Mercury's max surface temperature")
      0, 1, 2          -> println("This i as cold as it can physically get")
      in 300..699      -> println("This temperature is possible on Mercury")
      !in 0..300       -> println("This is pretty hot")
      earthSurfaceTemp() -> println("This is earth's average surface temperature")
      is Int             -> println("The given temperature is of type Int")
      else               -> {
        // Example of a multiline code block
        println("Default case")
      }
    }
```

As you can see, there are various ways to define the value(s) to compare against:

1. Fixed value (`700`)
2. Multiple fixed values (`0, 1, 2`)
3. Ranges (`in 300..699`)
4. Ranges with negation ("not in range") (`!in 0..300`)
5. Function call (`earthSurfaceTemp()`)
6. Type check (`is Int`)
7. Default case (`else`)

> **Note:** Kotlin adds a `break` to the end of each case implicitly so that you cannot introduce bugs due to a forgotten fallthrough case.

In fact, the `when` construct is even more powerful because it allows arbitrary conditions by leaving out the variable after the `when` keyword:

```
when {
      age >= 18 && !hasAccess -> println("Falsely rejected")
      age < 18 && hasAccess -> println("Falsely approved")
      else -> println("Correctly authorized")
```

```
}
```

Here, you can see the `age` variable is not mentioned in the `when` block's header as in `when (age)`. This way, you can use any boolean expressions on the left-hand sides of each `when` case, such as `age >= 18`.

Good Practice: `if` vs `when`

From a theoretical viewpoint, `if` and `when` are equally powerful. Coming from other languages, using `if` will feel more natural in most cases. Using `when` is definitely preferable when comparing against two or more distinct fixed values.

However, in aiming to write idiomatic Kotlin code, I'd encourage you to consider the `when` construct in more cases than the `switch` construct known from other languages. **In many cases, a `when` condition will be a more concise and readable alternative to a regular `if` condition**, especially whenever the right-hand sides are single expressions.

Quiz

Conditional control flow using `when`

1 Which left-hand side lets you check against a range of values inside a `when` condition?

Exercises

Rewrite all conditions from the previous lesson's exercise using `when`:

Complete the following code snippet to check whether...

1. The `age` variable is between 18 and 21 (both inclusive)
2. The `username` variable equals `"admin"` or `"system"`
3. The `number` variable is not equal to neither `17` nor `42`

...using only `when` conditions.

Problem | Solution

```
// Note: In this code widget, you have access to the predefined variables age, username, and

// Check age


// Check username


// Check number
```

Summary

Kotlin provides an `if` and a `when` construct for conditional control flow, which are equally expressive.

- `if` conditions are useful for simple if-then-else logic.
- `when` conditions are often more succinct if you have many branches, especially when checking against specific distinct values.

In the following lesson, you'll explore how to use `if` and `when` as expressions instead of statements.

Conditions as Expressions

Learn how to use conditions in Kotlin as expressions, including assigning different values to a variable depending on a condition.

WE'LL COVER THE FOLLOWING

- Expressions with `if`
  - Ternary Conditional Operator in Kotlin
- Expressions with `when`
- Quiz
- Exercises
- Summary

In Kotlin, both `if` and `when` can be used as *expressions* instead of *statements*. An *expression* is a piece of code that has a value, e.g. `"Kotlin"`, `42 * 17`, or `readInput()`. In contrast, a statement is a piece of code with *no* value, such as `fun foo() { ... }` or `while (active) { ... }`. In many programming languages, `if` and `when`/`switch` are statements. But in Kotlin, they are expressions! Let's explore how this works.

Expressions with `if`

Recall this listing from the lesson on `if` statements:

```
if (planet == "Jupiter") {
  println("Radius of Jupiter is 69,911km")
} else if (planet == "Saturn") {
  println("Radius of Saturn is 58,232km")
} else {
  println("No data for planet $planet")
}
```

If you think about it, it seems that the relevant data that changes here based on the condition is the radius. So let's store that in a variable by using an `if` expression:

```
val radiusInKm = if (planet == "Jupiter") {
        // Run code here...
        69911  // Last line defines the return value of this block
} else if (planet == "Saturn") {
        // Run code here...
        58232
} else {
        // Run code here...
        -1
}
```

Here, the entire `if-elseif-else` block has a value. This value for each branch is decided by the ***last line* in each branch**. You may run arbitrary code in each condition block but ultimately, the last line defines the value.

> **Note:** When assigning such an expression to a variable, the conditions must be *exhaustive*. This means that one of the condition blocks must match. The easiest way to achieve this is to have an `else` branch as fallback. Why must it be exhaustive? Because otherwise the `if` expression would not have a well-defined value in all cases, and thus it could not be assigned to a variable such as `radiusInKm`.

Ternary Conditional Operator in Kotlin

Kotlin has no separate language construct for a ternary conditional operator of the form `someCondition ? thenThis : elseThat`, which you may know from other languages. Instead, `if` expressions are used:

```
val category = if (age >= 18) "adult" else "child"
```

Note that you don't need curly braces around the condition blocks if they're a single expression.

Expressions with `when`

Naturally, `when` can be used as an expression in the same way:

```
val radiusInKm = when (planet) {
    "Jupiter" -> 69911
    "Saturn"  -> 58232
    else      -> -1
}
```

> In these simple examples, an `else` block is required for exhaustiveness. However, this is usually avoided in more advanced code by switching on values with a fixed finite set of possible values, such as enums or sealed classes in Kotlin. The compiler then infers whether you have covered all possible values.

Quiz

Conditional expressions with `if` and `when`

1 Which of the following correctly defines expressions vs statements? You can select multiple answers.

Exercises

In the following code listing, you are given a variable `priority`. Initialize a read-only variable `priorityText` and set it to a different value depending on the value of `priority`:

- `priority` is 1 => `priorityText` should be `"Trivial"`
- `priority` is 2 => `priorityText` should be `"Minor"`
- `priority` is 3 => `priorityText` should be `"Normal"`
- `priority` is 4 => `priorityText` should be `"Major"`
- `priority` is 5 => `priorityText` should be `"Critical"`
- Otherwise, it should be `"Unknown"`

Problem | Solution (if) | Solution (when)

```
// Insert your code here
```

Implement the previous exercise using both an `if` expression and a `when` expression. Which one seems better for this use case? Why?

Summary

- *Expressions* have a value. *Statements* do not.
- In Kotlin, `if` and `when` are *expressions*.
  - If you don't use the value, they work exactly like conditional *statements* in other languages.
- Conditional expressions are useful to assign a value directly instead of splitting up declaration and initialization.
  - Consider how you would solve the exercise above in a language like Java or C.

Congrats! You now know how to use conditions idiomatically in Kotlin.

In the following section, you will learn about the different types of collections

available in Kotlin, how to use them, and when to use which of them.

Lists

Learn how to create and use lists in Kotlin to store and manipulate multiple values in one data structure.

WE'LL COVER THE FOLLOWING

- Creating a List
- Read-only vs Mutable Lists
  - Why immutability?
- Working with Lists
  - Accessing Elements
  - Manipulating Elements
  - Overview: Kotlin's Collection Types
- Quiz
- Exercises
- Summary

Collections are a central feature of any programming language. They allow storing multiple values – such as your list of favorite books – in one convenient data structure which can then be stored and accessed with a single variable.

The fundamental collection types you'll master in the following lessons are:

- Lists
- Sets
- Arrays*
- Maps

*Arrays are not part of the actual Collections API but are included here because they're also a data structure for storing multiple elements.

Creating a List

Creating a List

Kotlin provides three convenient functions to initialize lists:

```
val siUnits = listOf("s", "m", "kg", "A", "K", "mol", "cd")                  // Creates reac
val quarks = mutableListOf("up", "down", "charm", "strange", "top", "bottom") // Creates muta
val physicists = arrayListOf("Albert Einstein", "Isaac Newton")              // Creates muta
```

All three functions allow you to add any number of elements (try it out!). The main difference between them is that `listOf` creates a **read-only list** whereas `mutableListOf` creates a **mutable list**. What does this mean? With read-only lists, you cannot add or remove elements from the list after it's created. With a mutable list, however, you can.

> The code example uses type inference so you can't see what the data types are. They are:
>
> | Function Call | Return Type |
> | --- | --- |
> | `listOf("", ...)` | `List<String>` |
> | `mutableListOf("", ...)` | `MutableList<String>` |
> | `arrayListOf("", ...)` | `ArrayList<String>` |
>
> You can read the type `List<String>` as "list *of* string".
>
> Mini-exercise: add explicit types to the code widget above and run it again.

Read-only vs Mutable Lists

The differentiation between read-only lists and mutable lists has the same underlying intention as differentiating between `val` vs `var`: immutability. At

this point, it's important to understand how `val` and read-only data structures differ. They essentially represent two levels of immutability:

- `val` prevents an initialized variable from being reassigned to another value. In other words, the memory address the variable points at cannot be changed; it will always point to the same memory address.
- Read-only collections prevent adding or removing elements from an existing collection but don't affect any variables that may point to these collections. In other words, they disallow changing values in the section of the computer's memory that stores the collection.

So how do these two play together?

	`val`	`var`
Read-only collection	Cannot reassign variable or change values stored in variable ("immutable")	Can reassign the variable (to a whole different collection) but can't overwrite the values in the original collection
Mutable collection	Cannot reassign variable but can freely change the values stored in the variable	Can both reassign the variable and change values in the collection it stores

Following the principle of immutability, use read-only collections and `val`s whenever you can.

> ### Why immutability?
>
> Using immutable data structures and `val`s allow you to better understand the data flow of your program. This facilitates enforcing invariants in your code, since you know that the contents of your variables cannot change. First, this avoids a variety of hard-to-fix bugs.

> Second, immutability is a fundamental principle of functional programming, which is a widely popular programming pattern.

Note that Kotlin's collections are not strictly immutable because there *are* intricate ways you could use to manipulate values in them. However, the Kotlin team is working on a library of actual immutable collections as opposed to just read-only collections.

Working with Lists

To work with lists, you must be able to access and manipulate its individual elements.

Accessing Elements

To access an element from a list, you use Kotlin's **indexed access operator** using square brackets:

```
val siUnit = siUnit[2]            // Gets 3rd element of siUnits
val quark = quarks[0]             // Gets 1st element of quarks
// val physicist = physicists[3]  // Would cause an error because there is no 4th element in
```

As you probably noticed, indexing starts at zero. So passing in *i* gives you the *i+1*-th element of a list. Consequently, the maximum index you can use without causing a runtime error is *i-1*.

Manipulating Elements

To manipulate the elements of a list, you also use the indexed access operator and reassign the value:

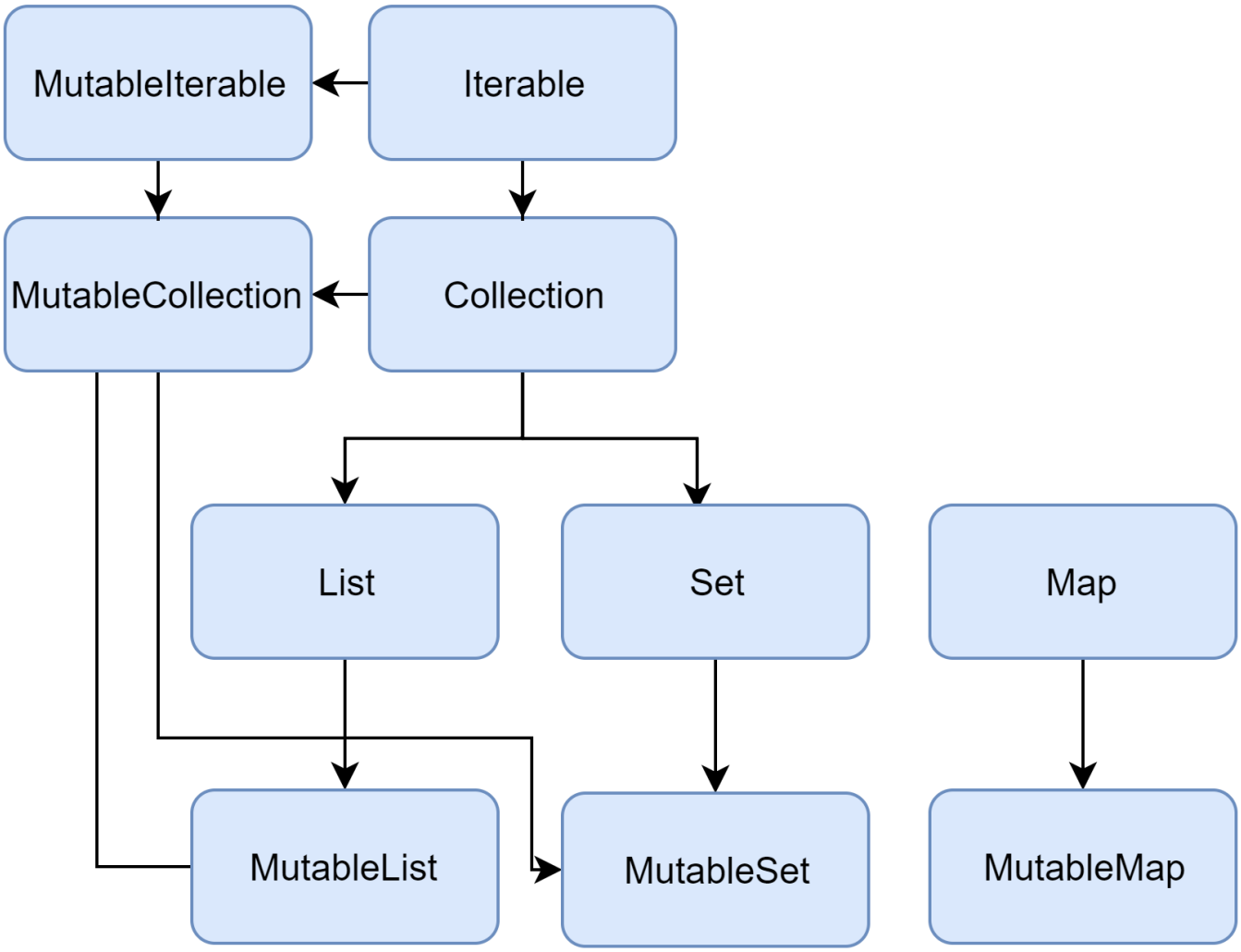
```
// siUnit[1] = "meter"          // Would cause error because siUnits is readonly
quarks[2] = "CHARM"             // Overwrites 3rd element of quarks
physicists[1] = "Marie Curie"   // Overwrites 2nd element of physicists
```

The syntax here is the same as for regular variable assignments, except for the additional index access in square brackets. Naturally, trying to overwrite a value of a read-only collection causes a compile-time error.

Compile-time error means that Kotlin won't allow the code to run. The compiler knows that it would fail, so it will reject compiling this program. It

might not appear so, but compile-time errors are your friend; they prevent nasty runtime errors that may require long debugging or, even worse, could crash a productive application. Kotlin's compiler is very good at catching a variety of potential mistakes.

> ### Overview: Kotlin's Collection Types
>
> In Kotlin, every collection type has a mutable and read-only variant, as portrayed in the following diagram:



Kotlin collections inheritance hierarchy. Each collection type has a readonly and a mutable variant. [Source: https://kotlinlang.org/docs/reference/collections-overview.html]

> `Iterable` and `Collection` are the more abstract interfaces from which lists and maps inherit. As you can see, `MutableList` is both a `List` and a `MutableCollection`, as indicated by the arrows. `Map`, which we'll cover later, doesn't inherit from `Collection` itself but is still part of the Collections API.
>
> **Note:** Arrays are not part of the Kotlin Collections API, so they are not

> **Note:** Arrays are not part of the Kotlin Collections API, so they are not included in this diagram. Arrays also do not have a read-only variant.

Quiz

Mutable and readonly lists

1 Which of the following create *mutable* lists? You can select multiple answers.

COMPLETED 0%

1 of 4

Exercises

In the following code widget, add three variables (use the highlighted variable names):

1. A read-only list of your best `childhoodFriends`
2. A mutable list of `books` you read in the last 6 months

After that, extend your code to complete the following challenges:

- Overwrite the 2nd favorite book in your list with any other book title
- If your list of books has 10 or more elements, print `"I devour books"`. If it has at least 5 elements, print `"I enjoy reading books from time to time"`.

Otherwise, print `"I'm more of a sporadic reader of books"`.

- Which condition construct is best for this use case?

Problem Solution

```
// Add your code here
```

> Collections are fundamental in programming and you will use them again in future exercises. This will help you solidify your knowledge of collections.

Summary

Like any higher-level programming language, Kotlin offers collections as data structures to store multiple values.

- What makes Kotlin different is that it clearly separates read-only and mutable collections.
- Lists can contain any number of elements. In mutable lists, elements can be changed, added, or removed.
- Particular elements of a list are accessed by index, using square brackets.

In the next lesson, you will learn about sets as an alternative data structure to store multiple elements.

Sets

Understand the data structure of sets, how to use them, and how they differ from lists.

WE'LL COVER THE FOLLOWING

- What is a Set?
- Creating a Set
- Accessing Elements in a Set
- Adding and Removing Elements
- Quiz
- Exercises
- Summary

Sets are linear data structures very similar to lists. However, they do have a few notable differences.

What is a Set?

Just like a list, a set is a linear data structure that contains multiple elements. However, sets are different from lists because:

- Sets have no concept of ordering, meaning there is no such thing as “the third element” in a set.
- Sets cannot contain duplicates; you cannot have a set that, for instance, contains `42` twice.

Both these properties of sets have implications for programming with them that you should be aware of when using them.

Creating a Set

The way to create a set should look familiar to you:

```
val awards = setOf("World's Best Programmer 2019", "Best Programming Language 2020")
val members = mutableSetOf("Susan", "Jake", "Jenny")
```

The impact of using read-only vs mutable sets is the same as for lists in the previous lesson. Try using read-only sets over mutable sets.

> If you already know your way around data structures, you may want to use any of the following functions to create a particular type of set:
>
> - `hashSetOf(...)`
> - `linkedSetOf(...)`
> - `sortedSetOf(...)`

Accessing Elements in a Set

Because there's no concept of ordering in a set, you *cannot* use an indexed access operator:

```
val members = mutableSetOf("Susan", "Jake", "Jenny")
println(members[0])  // Causes compile-time error
```

When using sets, you normally search for a subset of elements by trait or perform operations on the entire set. Because these concepts require a few more prerequisites, we'll come back to working with sets later.

For now, you may use Kotlin's utility functions such as `mySet.first()` to get an element from a set to try it out.

> **Note:** As per the definition of sets, the "first" element may be any of the values you passed in. You have no guarantee it's the first one you passed in.

After learning about loops in the next section, you'll be able to iterate over any collection to access each element.

Adding and Removing Elements

Adding and removing elements is similar to how you do it with lists:

```
val members = mutableSetOf("Susan", "Jake", "Jenny")
members.add("Greg")
members.remove("Jake")
members.clear()
members.addAll(setOf("Adam", "Eve"))
```

Although you cannot *yet* use sets in many ways, they are fundamental for practical applications. Oftentimes, we're not interested in getting the *i*-th element of a collection. Instead, we want to implement more complex queries on the elements, such as:

- "search all users that have logged in within the past 24h,"
- "find books that have not been returned on time," or
- "what are the permissions required for this action?".

For all of these, sets can be preferable to lists because there's no need for ordering.

Quiz

Mutable and Readonly Sets

1 How are sets *different* from lists? You can select multiple answers.

COMPLETED 0%

1 of 2

Exercises

In the following code widget:

- Create a read-only set of your `favoriteActors`
- Create a mutable set of your `favoriteMovies`
- Try to add a duplicate into one of your sets and see what happens.

Problem Solution

```
// Add your code here
```

Summary

Similar to lists, sets are a linear data structure of elements.

- Sets cannot have duplicates, so adding an existing element has no effect
- Sets have no concept of ordering, so you can't access an element by index
- You can create sets using `setOf` and `mutableSetOf`

Next, you will learn how to use arrays in Kotlin and how they differ from lists and sets as a data structure.

Arrays

Learn about the array data structure, how it differs from lists and sets, and how you can use them in Kotlin.

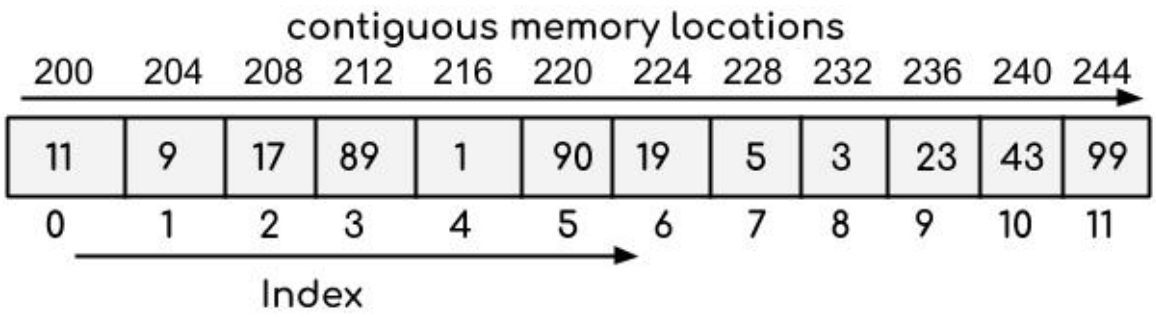
WE'LL COVER THE FOLLOWING

- What is an Array?
  - What's the Difference Between Arrays, Lists, and Sets?
- Creating an Array
- Working with Arrays
- Quiz
- Exercises
- Summary

Like lists and sets, arrays store multiple values in a linear data structure.

What is an Array?

An array is a basic data structure in all programming languages. It stores multiple elements by placing them consecutively in memory. This structure is represented in the following illustration:



Array data structure. Elements are stored contiguously in memory and index starts at zero. [Source: https://beginnersbook.com/2018/10/data-structure-array/]

What's the Difference Between Arrays, Lists, and Sets?

Sets are easy to distinguish from both lists and arrays based on their special properties: no ordering and no duplicates.

To understand the difference between lists and arrays, we'll have to look at the way their elements are stored and what implications this has for us as developers.

While arrays store their values in one consecutive part of computer memory, list and set elements may be spread across memory. This has several implications:

- **Arrays are a fixed-length data structure.** This is because, after the array is initially created, the next consecutive place in memory may be taken by another value. Thus, no more consecutive values can be added to the array. Not being able to add or remove elements distinguishes them from *mutable* lists.
- Differences between arrays and read-only lists exists only on a lower level, i.e., in the exact API they provide, the storage structure in memory, and their resulting runtime performance.

Finally, there's no differentiation between read-only and mutable arrays in Kotlin. Array elements can be overwritten but no elements can be added or removed, as they're fixed in length.

> **Note:** Recall that arrays are not part of Kotlin's Collections API. But they have a similar data structure, so they're mentioned here. All actual *collections* have read-only and mutable variants.

Data structure	Read-only available?	Ordered?	Duplicates allowed?	Memory storage
List	Yes	Yes	Yes	Non-sequential
Set	Yes	No	No	Non-

Set	Yes	No	No	sequential
Array	No	Yes	Yes	Sequential

Creating an Array

Kotlin's API is very consistent when it comes to helper functions so the way to create an array should look familiar:

```
val priorities = arrayOf("HIGH", "MEDIUM", "LOW")
```

The `arrayOf` function accepts any number of arguments and returns an `Array<Something>`. In the example, it returns an `Array<String>` (reads "array of string").

Working with Arrays

While you can't add or remove elements from an array, you *can* access and manipulate all its elements using the indexed access operator:

```
val priorities = arrayOf("HIGH", "MEDIUM", "LOW")
println(priorities[1])
priorities[1] = "NORMAL"
println(priorities[1])
println(priorities.size)
```

As you can see, the way to access and manipulate existing elements is consistent across all linear data structures.

Quiz

Arrays in Kotlin

1

Which of the following are properties that make *arrays* different from *sets*? You can select multiple answers.

COMPLETED 0%

1 of 3

Exercises

Initialize an array of `programmingLanguages` you're familiar with.

- Print the first element of your array to the console
- Overwrite the first element by the next programming language you want to learn
- Print the first element of your array to the console again to verify it was changed

Problem Solution

```
// Add your code here
```

Summary

Arrays are a linear data structure that store multiple values in memory sequentially.

- In contrast to sets, arrays are ordered, may contain duplicates, and store

elements sequentially in memory.

- In contrast to lists, arrays are fixed-length because they store elements sequentially, and are always mutable.
- You can create arrays using `arrayOf(...)`.

The next lesson introduces maps, the first non-linear collection data structure.

Maps

Understand the map data structure, its use cases, and how to put it to use in Kotlin.

WE'LL COVER THE FOLLOWING

- Creating a Map
- Pairs and Infix Functions
  - Creating a Mutable Map
- Working with Maps
  - Accessing Entries of a Map
    - Mini Challenge
  - Manipulating Entries in a Map
- Quiz
- Exercises
- Summary

A map is a collection of keys associated with values. This is a concept you may also know as *dictionaries* (e.g., Python) or *associative arrays* (e.g., PHP). Each key is associated with one unique value. This value can have any type. In particular, it may be a collection, such as a list or a nested map.

Creating a Map

Once again, Kotlin provides a convenient way to initialize (read-only) maps:

```
val grades = mapOf(
  "Kate" to 3.9,
  "Jake" to 3.4,
  "Susan" to 3.5
)
```

This map associated the key `"Kate"` to the value `3.9`, `"Jake"` to `3.4`, and so on.

> ## Pairs and Infix Functions
>
> The `to` function is a so-called *infix function*, a concept which we'll cover later in this course. It returns a `Pair`, which is a simple data structure containing exactly two values. Therefore, you can also initialize the array above using Pairs directly:
>
> ```
> val grades = mapOf(
>  Pair("Kate", 3.9),
>  Pair("Jake", 3.4),
>  Pair("Susan", 3.5)
> )
> ```
>
> Kotlin makes this initialization more convenient and readable by providing the `to` function.

From now on, it will be important to understand the terms map, entry, key, and value. In the example above:

- The `grades` variable is the *map*.
- This map contains three *entries*, and each *entry* has a key and a value.
- The first entry's *key* is `"Kate"`, and its *value* is `3.9`.

Creating a Mutable Map

As you may expect by now, Kotlin offers `mutableMapOf` to create a mutable map:

```
val enrollments = mutableMapOf(
  "Kate" to listOf("Maths", "Engineering"),
  "Jake" to listOf("CS", "Bioengineering", "Psychology"),
  "Susan" to listOf("Engineering", "Psychology")
)
```

By using a mutable map, you can add and remove entries from the map, as we'll cover later in this lesson.

Working with Maps

Doing useful computations on maps requires accessing and manipulating its entries.

Accessing Entries of a Map

Accessing an entry of a map is again achieved using the *indexed access operator*. But this time, we'll pass in the key of the entry we want to retrieve:

```
val enrollments = mutableMapOf(
  "Kate" to listOf("Maths", "Engineering"),
  "Jake" to listOf("CS", "Bioengineering", "Psychology"),
  "Susan" to listOf("Engineering", "Psychology")
)

val classes = enrollments["Kate"]
println(classes)   // ["Maths", "Engineering"]
```

Since you can't access the map entries by index (only by key), **maps are unordered collections**, similar to sets. Note that the value itself may again be a collection, so you could use the indexed access operator on it. You can nest collections arbitrarily deep.

> Mini Challenge
>
> Try to access Jake's third enrolled class, starting with the `enrollments` map.

If no entry with the given key exists, the indexed access operator returns `null`. To avoid nullability in your code, you can use `getOrDefault` and `getOrElse`. With `getOrDefault`, you pass in an additional default value which is used in case the key doesn't exist. With `getOrElse`, you can pass in a block of code that computes the value to use in case the key doesn't exist:

```
val enrollments = mutableMapOf(
  "Kate" to listOf("Maths", "Engineering"),
  "Jake" to listOf("CS", "Bioengineering", "Psychology"),
  "Susan" to listOf("Engineering", "Psychology")
)

val marcusClasses = enrollments.getOrDefault("Marcus", emptyList())
println(marcusClasses)   // [], not null

val simonsClasses = enrollments.getOrElse("Simon", {
    /* compute... */
    emptyList()
})
println(simonsClasses)   // [], not null
```

Both work almost the same, but `getOrElse` lets you compute more complex expressions inline, whereas, with `getOrDefault`, the desired value should be readily available.

Manipulating Entries in a Map

Assuming you have a *mutable* map, you can add, remove, and manipulate entries. To add a new entry, you use `put`:

```
enrollments.put("Marcus", listOf("Maths", "CS"))

// Can be written more idiomatically as:
enrollments["Marcus"] = listOf("Maths", "CS")
```

The `put` method accepts a key and value, similar to when you first initialize a map. The idiomatic way to write this is using square brackets, consistent with indexed access syntax.

Second, you can remove an entry using `remove` by passing the key of the entry to remove:

```
enrollments.remove("Jake")
```

Although this is by far the most common way to remove an entry, you can also pass an additional value. In that case, an entry is only removed if both its key and value match your arguments.

Third, you can overwrite the value of an existing entry (in case of a mutable map):

```
enrollments["Susan"] = listOf("CS", "Psychology")
```

This overwrites the value for the entry with key `"Susan"`.

> Note that the syntax is the same as for adding a new element. Therefore, if the given key (`"Susan"`) doesn't exist, this will add a new map entry.

If the map stored mutable lists of enrollments, you could also add or remove them from the existing classes instead of overwriting the entire list. For instance, using `enrollments["Susan"].add("Business Administration")`.

Quiz

Mutable vs Read-only Maps

1 Which of the following statements are correct?

COMPLETED 0%

1 of 4

Exercises

First, create a map with the past two years and current year as keys. For instance, if it's 2020, use 2018, 2019, and 2020.

Then, assign a list of things you achieved that year to each key, or assign them to whatever you were grateful for that year (maybe a holiday, gatherings with friends, a career milestone, learning a new skill, etc.).

After that, recap all main map operations using your code above:

- Make your map a mutable map
- Add a new entry for three years ago
- Remove the entry for the current year
- Make the list of achievements or things you're grateful for mutable
    - Then add one more element to the existing list for last year

Hint: You can print each intermediate map using `println(myMap)` to sanity-check your code.

Problem Solution

```
// Add your code here
```

Summary

Maps are a fundamental data structure that you will find throughout real-world code. Therefore, mastering maps is crucial.

- Maps contains *entries* that map *keys* to *values*.
- Basic maps are created using `mapOf` or `mutableMapOf`.
    - Again, prefer read-only maps over mutable ones.
- Kotlin provides `to` for creating map entries in readable way (`myKey to myValue`).
- You can add entries to a mutable map using `myMap[key] = value`.
    - Overwriting an existing entry works the same way (if `key` already

exists).

- You can remove entries from a mutable map using `myMap.remove(key)`.

Congratulations, you are now familiar with the most widely used collection types in Kotlin. In the next section, you will learn how to use loops for control flow.

`while` Loops

Get started with Kotlin's loop constructs for repeating blocks of code.

WE'LL COVER THE FOLLOWING

- `while` Loops
- `do-while` Loops
- Quiz
- Summary

Kotlin offers the three standard types of loops:

- `while` loops
- `do-while` loops
- `for` loops

As a programmer, loops in Kotlin will feel familiar quickly. Let's look at a few examples, starting with `while` and `do-while` loops.

`while` Loops

Kotlin's `while` loops have exactly the same syntax and semantics as `while` loops in Java or C:

```
val input = 17
var sqrtApprox = 1.0
var delta = 1.0

while (delta > 0.001) {  // `while` block is repeated until error gets below threshold
  sqrtApprox = 0.5 * (sqrtApprox + input / sqrtApprox)
  delta = Math.abs((input - sqrtApprox*sqrtApprox) / (2*sqrtApprox))
}

println("sqrt($input) is approximately $sqrtApprox")
```

The `while` keyword is followed by a condition. The loop will repeat its code block until this condition evaluates to `false`.

This code example approximates the square root of the `input` up to an error margin of at most `0.001`.

`do-while` Loops

The `do-while` loop checks its condition *after* each iteration, whereas the regular `while` loop checks it *before* each iteration. Consequently, a `do-while` loop always performs at least one iteration. This is why it's typically used in cases where that first iteration gets some initial data, e.g. user input:

```
do {
  print("> ")
  val input = getUserInput()
  println("User typed: $input")
} while (input != ":q")
```

The advantage here is that you can limit the scope of the `input` variable to the loop block, which is not possible using a regular `while` loop. It's good practice to minimize the scopes of your variables.

Quiz

Kotlin's `while` Loops

1 What's the output of the following program?

```
println("Output: ")
while(false) {
  println("#")
}
```

> **Note:** Exercises follow in the next lesson on `for` loops.

Summary

Kotlin provides two slightly different `while` loop structures:

- The regular `while` loop checks its condition *before* each iteration.
- The `do-while` loop checks its condition *after* each iteration.
- Generally, `while` loops are used when the number of iterations is not fixed.

In the next lesson, you will explore Kotlin's `for` loops as an alternative to `while` loops, and learn when to prefer which type of loop.

`for` Loops

Learn about Kotlin's `for` loops to iterate over any iterable structure, and how to build ranges for iteration.

WE'LL COVER THE FOLLOWING

- `for` Loops
- `for` Loops vs `while` Loops
- Quiz
- Exercises
- Summary

In contrast to `while` loops, Kotlin's `for` loops work a bit differently than you may be used to from other languages.

`for` Loops

Kotlin's `for` loops are different from many other languages because they're used in the same manner as *for-each loops* (or *for-in loops*). Thus, you always give an *iterable* to the `for` loop over which it iterates:

```
for (number in 1..5) println(number)        // 1, 2, 3, 4, 5

for (number in 1 until 5) println(number)   // 1, 2, 3, 4

for (number in 1..5 step 2) println(number) // 1, 3, 5

for (number in 5 downTo 1) println(number)  // 5, 4, 3, 2, 1

for (number in 5 downTo 1 step 2) println(number)  // 5, 3, 1

for (char in 'a'..'c') println(char)        // 'a', 'b', 'c'

for (planet in planets) println(planet)     // "Jupiter", "Saturn", ...

for (char in "Venus") println(char)         // 'V', 'e', 'n', 'u', 's'
```

A few things to note here:

- *Ranges* are commonly used for basic loops, e.g., `1..5`.
  - There are utilities to construct more complex ranges, e.g., `downTo` and `step`.
- Other common iterables are collections, e.g., `planets`.
- Kotlin's strings are also iterable.

> **Note:** You can iterate over any object that implements `Iterable` using a `for` loop.

`for` Loops vs `while` Loops

As with `if` and `when`, you have multiple language constructs for one concept: repeating blocks of code. So when would you prefer `for` over `while` and vice versa?

- `while` loops are used when the *number of iterations is not known in advance*.
  - Consider again the approximation algorithm above. It may take any number of iterations to get below the required error margin because it's not a fixed number.
- `for` loops are superior when the *number of iterations* ***is*** *known in advance*.
  - In other words, the number of iterations is fixed because it's equal to the length of the given iterable.

Quiz

Kotlin's `for` loops

1 Which of the following can you iterate over using a `for` loop? You can select multiple answers.

COMPLETED 0%

1 of 3

Exercises

Implement a program that produces the following output *using loops*:

```
@
@@
@@@
@@@@
@@@@@
```

Hints:

- You can nest loops, meaning that you can write a `for` loop inside another `for` loop.
- You can use `print` instead of `println` to avoid going to a new line after printing

Problem Solution

```
// Add your code here
```

Next, extend your program to produce the following output:

```
@ @@@@@
@@ @@@@
@@@ @@@
@@@@ @@
@@@@@ @
```

Problem Solution

```
// Add your code here
```

Finally, write a program that prints a Christmas tree:

```
   @
  @@@
 @@@@@
@@@@@@@
@@@@@@@@@
   @@@
```

Problem Solution

```
// Add your code here
```

Summary

Kotlin offers `for`, `while`, and `do-while` loops, each of which fits the bill for different use cases:

- Use `for` loops for a fixed number of iterations.
  - Kotlin offers many utilities that work well with `for` loops, such as *ranges* (`1..10`), `downTo` and `step`.
- Use `while` for a non-fixed number of iterations.
  - Use `do-while` if the first iteration initializes some data used in the

loop body.

Congrats, you now know how to use loops in Kotlin! In the next section, you will learn lots about functions in Kotlin, including using default values, infix functions, extension functions, and more.

Basic Functions

Learn how to modularize chunks of code into functions and how to call your functions in Kotlin. Understand the difference between function signatures and function declarations.

WE'LL COVER THE FOLLOWING

- Function Signatures
- Declaring a Function
- Calling a Function
  - Function Types
- Quiz
- Exercises
  - Multiplication Table
  - Greatest Common Divisor
  - Least Common Multiple
- Summary

Functions separate useful chunks of code into a named entity you can reference in your code. Along with variables, they are the absolute fundamental language construct to avoid code duplication.

Function Signatures

A *function signature* defines a function's name, inputs, and outputs. In Kotlin, it looks like this:

```
fun fibonacci(index: Int): Long
```

This signature defines a function named `fibonacci` with one input parameter named `index` of type `Int` and a return value of type `Long`. In other words, you give the function an integer and get back a `Long`. This function

implements the well-known Fibonacci sequence.

Declaring a Function

In a *function declaration*, both a function signature and a *function body* are required:

```
fun fibonacci(index: Int): Long {
  return if (index < 2) {
    1
  } else {
    fibonacci(index-2) + fibonacci(index-1)  // Calls `fibonacci` recursively
  }
}
```

The function body is a code block (thus surrounded by curly braces) and defines what the function does. The `return` keyword defines the return value of the function call.

> **Note:** Notice how the `return` keyword is moved outside of the condition blocks. This is possible because the `if` block is an expression and therefore has a value.

Calling a Function

Once you have declared a function, you can call it by passing in a value for each required input parameter:

```
val output = fibonacci(6)
println(output)
```

This *function call* sets the `index` parameter to `6` and runs the function with that input. For the `output` variable, the Kotlin compiler infers the type `Long` based on the function's return type.

> ### Function Types

Functions have well-defined types in Kotlin. For instance, the `fibonacci` function has type `(Int) -> Long`, which means it accepts an `Int` as input and returns a `Long`. In other words, it's "a function from Int to Long". This concept becomes important once you get into *functional programming* in Kotlin.

Quiz

Working with Functions in Kotlin

1 Which of the following is a valid function signature in Kotlin?

COMPLETED 0%

1 of 3

Exercises

At this point, you've mastered enough concepts to solve basic algorithmic exercises. Such exercises are by far the most effective way to really grasp the language and use it confidently.

Note: This lesson contains several exercises, and these may take longer than the ones from previous lessons.

> This is because you are now at a point where you can solve actual computing problems. Please take your time to complete these exercises to build a strong foundation for the following lessons.

Multiplication Table

Implement a function that prints a multiplication table. As inputs, it accepts two ranges: one for the rows to use, and one for the columns.

Call your function to print a multiplication table with rows `1..5` and columns `1..8`.

Problem Solution

```
// Add your code here
```

Hints:

- Use the type `IntRange` for the input parameters
- You can use the standard library function `System.out.format("%-8d", value)` to output a left-aligned value with a padding of 8 chars. This is necessary to print a proper table.

Greatest Common Divisor

Write a function that computes the Greatest Common Divisor (GCD) of two given integers. Call your function to find the GCD of (54, 24), (81, 153), and (137, 73).

Problem Solution

```
// Add your code here
```

Expected output:

- `gcd(54, 24) = 6`
- `gcd(81, 153) = 9`
- `gcd(137, 73) = 1`

Least Common Multiple

Write a function that computes the Least Common Multiple (LCM) of two given integers. Sanity-check your function by calling it with several example inputs. Try to vary the inputs and explore edge cases (e.g. cases where `lcm(a, b) == a`, `lcm(a, b) == b`, or `lcm(a, b) == a*b`).

Problem Solution

```
// Add your code here
```

Hint: There is a way to calculate the LCM based on the GCD so you can reuse your GCD function for this exercise.

Summary

Functions are possibly the most crucial concept to understand as they are one of the main building blocks in your code. Most day-to-day programming consists of adding, splitting up, removing, and otherwise working with functions.

- Kotlin uses the `fun` keyword to introduce functions.
- A function consists of function signature and function body.
  - The signature defines a function's name, parameters, and return type.
  - The function body defines what the function does.
- Functions in Kotlin have well-defined types that contain parameter types and return type.

In the next lesson, you will finally learn how to write your own `main()` function to implement an executable Kotlin file. So far, this part of the code has been hidden in the code listings to facilitate your learning.

The main() Function

Learn about the special main function as the entry point into your program.

WE'LL COVER THE FOLLOWING

- Anatomy of a `main` Function
- Quiz
- Exercises
- Summary

Every application requires an entry point where program execution is started. In most languages, including Kotlin, this is called the `main` function.

> Kotlin also allows writing *script files* (`.kts` file ending) which don't require an explicit `main` function, similar to Bash or Python scripts. When talking about an "application" above, such scripts are excluded.

In fact, even the code listings here on Educative require a `main` function, although you have not seen one yet. Up until this point, a `main` function was always hidden in code surrounding the code you saw in the listings. This is what enabled the code examples to run.

Anatomy of a `main` Function

The `main` function in Kotlin is simple:

```
fun main() {
  // Your code here...
}
```

It has the simplest possible signature: no inputs and no outputs*. This has been the case since Kotlin 1.3. Before that, an input parameter was required for command-line arguments:

```
fun main(args: Array<String>) {
  // Your code here...
}
```

Oftentimes, we don't need the command-line arguments in our code. In these cases, we can use the simpler signature.

> *To be more specific, "no output" in Kotlin means that it returns `Unit`. However, this makes no difference for us at this point.

Quiz

The `main` Function

1 Which types of programs requires a `main` function as entry point?

Exercises

Populate the following code listing with your GCD implementation and a `main` function. In your `main` function, print the `gcd(93, 27)`.

Problem | Solution

```
// Add your code here -- no hidden code in this listing to help you this time :)
```

Hint: You can call your GCD function from within the `main` function.

Summary

Every Kotlin application you will write requires an entry point in the form of a `main` function.

- You can use the simple signature `fun main()` in most cases.
- To have access to any command-line with which your application was started (e.g., for configuration), you can use `fun main(args: Array<String>)`.

Next up, you'll learn about Kotlin's shorthand notation for functions which can be a great tool to write concise but readable code.

Shorthand Notation for Simple Functions

Learn when and how to write functions in shorthand notation for concise and readable code.

WE'LL COVER THE FOLLOWING

- Refactoring a Function
- Introducing Shorthand Functions
  - Combining with Type Inference
- Quiz
- Exercises
  - Fibonacci
  - GCD
  - LCM
- Summary

Functions that simply return the result of a single expression can be abbreviated in Kotlin.

Refactoring a Function

Consider the following function:

```
fun isValidUsername(username: String): Boolean {
  if (username.length >= 3) {
    return true
  } else {
    return false
  }
}
```

How would you refactor it? Can you make it more concise and improve its readabilty?

You could improve this in several steps, e.g., by moving out the `return`

You could improve this in several steps, e.g., by removing out the `return` keyword and using `if` as an expression. This particular example is even simpler and can be rewritten as:

```
fun isValidUsername(username: String): Boolean {
  return username.length >= 3
}
```

This is the simplest form this function could take in many other languages such as Java or C++. The function body of `isValidUsername` **consists only of a `return` keyword followed by an expression**.

But in Kotlin, you can go one step further.

Introducing Shorthand Functions

For *single-expression functions* like this, Kotlin allows a shorthand notation:

```
fun isValidUsername(username: String): Boolean = username.length >= 3
```

The function body with curly braces is now replaced by an assignment, the same way you would assign a variable. This makes the code more succinct and, once you're used to this syntax, it improves readability by removing boilerplate (curly braces and `return`).

Combining with Type Inference

At this point in the refactoring, you can go one step further and make use of Kotlin's type inference:

```
fun isValidUsername(username: String) = username.length >= 3
```

This only works with shorthand notation. The compiler now infers that the `isValidUsername` function returns a `Boolean` (the same way it can infer it for variables).

> This is not to say you should always remove the explicit return type. You should evaluate if it actually helps readability or not. In this simple example above, it's immediately obvious to developers that it returns a

`Boolean`, even from the function name.

In other cases, it may not be so clear. You should keep in mind that, without an explicit return type, you may accidentally change a function's return type by changing the function's body.

Quiz

Shorthand notation for functions

1 Which of the following functions could be *directly* written in shorthand notation (without further refactoring)? Why?

COMPLETED 0%

1 of 2

Exercises

Fibonacci

1. Implement a function that returns the *n*-th Fibonacci number.
2. In the next step, compress your function body into a single expression (including the `return` keyword).
3. Finally, transform it into shorthand notation.

4. Call your function with several inputs in a `main` function

(Of course you can do this all at once, but it's easier to go step by step).

Problem Solution

```
// Add your code here
```

GCD

Rewrite your GCD function so that you can fit the entire function into a single readable line of code using shorthand notation.

Problem Solution

```
// Add your code here
```

LCM

Rewrite your LCM function in the same way. You may still reuse your GCD function by pasting it here.

Problem Solution

```
// Add your code here
```

Summary

Kotlin allows a very concise function notation in some cases.

- The function body must contain only a `return` followed by a single expression.
- It can then be defined as `fun foo() = 42`.

    - The return type can be omitted due to type inference.

- The shorthand can improve or hamper readability, depending on the function.

In the following lesson, you'll set default values on function parameters to increase your functions' usability and flexibility.

Default Values & Named Parameters

Discover how to use default parameter values to increase the flexibility of your functions and how to call functions using named parameters.

WE'LL COVER THE FOLLOWING

- Default Parameter Values
  - Named Parameters
  - In a World Without Default Values...
- Quiz
- Exercises
- Summary

In Kotlin, you can define default values for function parameters to make them optional.

Default Parameter Values

Let's say you have a function that joins a collection of strings into a single string:

```
fun join(strings: Collection<String>, delimiter: String) = strings.joinToString(delimiter)
```

For instance, `join(listOf("Kotlin", "Java"), " > ")` would return `"Kotlin > Java"`.

With this function definition, you always have to pass in a `delimiter`. But most commonly, you'll want to have a comma as delimiter, so it makes sense to set this as the default value:

```
fun join(strings: Collection<String>, delimiter: String = ", ") = strings.joinToString(delimi
```

Default values are defined by simply adding an assignment to the parameter definition. This way, you now have different ways to call this function:

```
fun join(strings: Collection<String>, delimiter: String = ", ") = strings.joinToString(delimi

fun main() {
  val planets = listOf("Saturn", "Jupiter", "Earth", "Uranus")

  val joined1 = join(planets, " - ")
  val joined2 = join(planets)

  println(joined1)
  println(joined2)
}
```

You're still free to use a custom delimiter by passing in a value, but you can now omit the delimiter to simplify the function call. Think of these as **optional parameters** that can majorly increase the flexibility of your functions (i.e., of your API).

Kotlin's own `joinToString` function makes heavy use of default values because it's a prime example for their usability. You *can* pass in a delimiter, prefix, postfix, a transformation and more. But you can also call it with any subset of these or without any arguments.

> **Note:** Although the Kotlin compiler *could* infer the type of the `delimiter` parameter from the default value, you are required to define the type explicitly. This is because function signatures define an API that must be well-defined and that you should have put thought into (e.g., using an abstract type like `Collection` or `Iterable` instead of `List` to increase the function's flexibility).

Named Parameters

Using named parameters, there are even more ways you can call the `join` function:

```
fun join(strings: Collection<String>, delimiter: String = ", ") = strings.joinToString(delimi

fun main() {
  val planets = listOf("Saturn", "Jupiter", "Earth", "Uranus")

  val joined1 = join(planets, delimiter = " - ")
  val joined2 = join(strings = planets)
  val joined3 = join(strings = planets, delimiter = " - ")
  val joined4 = join(delimiter = ", ", strings = planets)

  println(joined1)
  println(joined2)
  println(joined3)
  println(joined4)
}
```

When you call a function in Kotlin, you can explicitly state the name of any input parameter. If you do this for a parameter, all the following ones have to be named as well.

Named parameters improve readability whenever the meaning of arguments is not clear from the function call itself, which is often the case when a function has many input parameters.

Note that you can change the order of your arguments when using named parameters.

In a World Without Default Values...

In a programming language without default values, you need to use *function overloading* to implement an equivalent `join` function. In Java for instance:

```
String join(Collection<String> strings, String delimiter) { ... }
String join(Collection<String> strings) { return join(strings, ", "
) }  // Overloaded function defines default value
```

Obviously, this requires more code, and it takes longer to identify the default values. More importantly, this so-called *telescoping (anti-)pattern* requires an exponential number of overloaded functions for more parameters with default values.

Quiz

Optional and Named Parameters

1 Which of the following function declarations uses default values correctly?

COMPLETED 0%

1 of 3

Exercises

Adjust your multiplication table function from the lesson “Basic Functions” by making both the `rows` and `columns` parameter optional with a default value. Then call your function:

1. by passing arguments for both parameters
2. by passing only a `rows` argument
3. by passing only a `columns` argument
4. by passing no argument

Also try passing your arguments in different orders using named parameters.

Problem Solution

```
// Add your code here
```

Summary

You can make function parameters optional by defining a default value.

- Add the default values to the function signature, e.g. `fun makePizza(toppings: List<String> = emptyList())`.
  - You still have to define the parameter types explicitly.
  - There is no need to overload functions for this use case, which saves lots of boilerplate code.
- Use named parameters to pass any subset of optional parameters or to improve readability at call site.

The next lesson introduces Kotlin's concept of extension functions, a powerful feature to extend the API of existing and third-party entities.

Extension Functions

Understand Kotlin's extension functions, how to define them, and why they are a fundamental feature of Kotlin.

WE'LL COVER THE FOLLOWING

- Defining an Extension Function
  - Extensions are Resolved Statically
- Real-World Examples of Extension Functions
- Quiz
- Exercises
  - An Extension on `Date`
  - An Extension on `String`
- Summary

Extension functions allow you to attach new functions to existing types. *This includes ones that you don't own*, such as `Int`, `String`, any predefined Java class such as `Date`, `Instant`, and any type imported from a library.

Typically, this is done using utility classes that encapsulate a series of utility functions. You may be familiar with class names such as `StringUtils` or `DateUtils`. In Kotlin, missing capabilities of a class you want to use can be patched using extension functions instead.

Defining an Extension Function

Let's say you want to use Java's old `Date` class (instead of the `java.time` package). In your application, you want to add a number of days to a given `Date`. Ideally, you'd want to write the following:

```
// Goal
val now = Date()
val tomorrow = now.plusDays(1)  // No such method yet
```

However, Java's `Date` class has no such `plusDays` method. Traditionally, you would create a helper function that allows you to write `val tomorrow = plusDays(now, 1)`, but this call syntax has several drawbacks:

- It doesn't read as naturally
- The function is not scoped to `Date` objects
  - Thus, autocomplete is a lot less helpful

Fortunately, Kotlin's extensions can achieve what we want. A `plusDays` function that fulfills the requirements above can be written as follows:

```
fun Date.plusDays(count: Int) = Date(this.time + count * 86400000L)
```

> **Note:** The function signature contains the `Date` class in front of the function name. This implicitly makes `Date` the first parameter of the function, and you can access it inside the function body as `this`.

The actual implementation simply adds the correct number of milliseconds to the original date to add `count` days.

With this function defined as above, Kotlin now allows you to call this method as if it were defined in the class itself:

```
val now = Date()
val tomorrow = now.plusDays(1)
```

> Under the hood, Kotlin transforms this to a regular utility function. This is necessary because the generated Java bytecode doesn't allow for adding a new method to an existing class.

Extensions are Resolved Statically

> These following few paragraphs require knowledge on object orientation.

Although extensions look like regular methods being called on objects, they are resolved statically, not dynamically. What does this mean? Consider the following example:

```
fun Number.print() = println("$this is a Number")
fun Int.print() = println("$this is an Int")

val num: Number = 17
num.print()  // Prints "17 is a Number"
```

`Int` is a subclass of `Number`. If `print` were a regular method in the `Number class` overwritten in the `Int` class, then to method call in the last line would be resolved dynamically at runtime. Since `num` is an `Int` at runtime, it would call `Int.print`.

This is not the case with extension functions. The call in the last line is resolved statically at compile time, and since `num` is declared to have type `Number`, the compiler will transform the `print` call to calling `Number.print`.

If you ever have a bug where the wrong extension is called, knowing these details can be essential.

> **Note:** If the type you extend already has a regular member function with the same name as your extension, the member function takes precedence.

Real-World Examples of Extension Functions

As mentioned, extension functions are a fantastic way to improve clunky parts of any APIs that you're using. Since Kotlin has become the de-facto standard for Android development, let's look at a simple example from the Android domain.

Android domain.

On Android, when you're inside an `Activity` class (imagine a "screen" in an Android app), and you want to get its `ViewModel` (imagine "data provider" for that activity), you usually write this:

```
val viewModel = ViewModelProviders.of(this).get(MyViewModel::class.java)
```

This is an Android API. However, let's make it so that the same can be achieved using this piece of code:

```
val viewModel = getViewModel(MyViewModel::class)
```

To do so, all you need is an extension function on the `Activity` which handles getting a `ViewModel` from the context of the activity:

```
fun <T : ViewModel> FragmentActivity.getViewModel(modelClass: KClass<T>): T {
  return ViewModelProviders.of(this).get(modelClass.java)
}
```

For now, don't worry about the generic types in angle brackets. But on a high level, you can see the extension is more Kotlin-friendly by expecting a `KClass` as parameter instead of a Java `Class`. This way, you can pass in `MyViewModel::class` instead of `MyViewModel::class.java`.

> **Takeaway:** If you ever feel like the API you're using is unnecessarily verbose, think about **encapsulating the boilerplate in an extension function**. For instance, if you always call a function with certain values for some parameters, you could create an extension function with default values for these to reduce the number of arguments.

Quiz

Extension Functions

1

What can extension functions be used for? You can select multiple answers.

COMPLETED 0%

1 of 3

Exercises

An Extension on `Date`

Implement an extension function on the `java.time.Date` which adds a given number of minutes to the original date. You should be able to call this extension as follows:

```
val now = Date()
val later = now.plusMinutes(5)
```

Hint: You'll have to import `java.util.Date` by adding the following at the top of your code:

```
import java.util.Date
```

Problem Solution

```
// Add your code here
```

An Extension on `String`

An Extension on `String`

In this exercise, implement an extension on `kotlin.String` which removes all occurrences of a given string from the original string. It should provide the following API:

```
val original = "Extension functions allow you to attach new functions to e
xisting types *including ones that you don't own*"

val edited1 = original.removeAll("*")  // Should remove both *
val edited2 = original.removeAll("you")  // Should remove all "you"s
```

Problem Solution

```
// Add your code here
```

Hint: First, figure out how to remove a substring once. After that, figure out how to repeat the removal step for as long as necessary.

Hint: If you're using an IDE with autocomplete, take a look at all the methods Kotlin offers on `String`s. If you're doing this in the browser: the `replace` method may be useful.

Summary

Extension functions in Kotlin are a prevalent part of Kotlin applications in practice and in Kotlin's standard library.

- Extensions allow you to add new functionality to a base API.
- The declaration of extensions is not bound in location, meaning extensions can be defined anywhere, not just inside the receiver class' body.
- Extensions can be called directly on the corresponding types' objects.
  - This enables code completion, improves code structure, and limits the scope of extensions.
  - Under the hood, Kotlin generates a good old utility class. In the generated Java bytecode, you cannot actually add new functions to

types from outside.

- In the following lessons, the concept of extension functions will be explored further with infix functions and operators.

In the next lesson, you'll learn how to implement and use a special class of functions called *infix functions*.

Infix Functions

Discover what infix functions are and how you can implement your own to write more readable Kotlin code.

WE'LL COVER THE FOLLOWING

- What are Infix Functions?
- Implementing an Infix Function
  - Restrictions for Infix Functions
- Quiz
- Exercises
- Summary

Infix functions are a great tool to improve readability of function calls in certain cases.

What are Infix Functions?

In programming, function calls are typically written in prefix notation, meaning that the function name is in front, followed by its arguments in parentheses:

```
gcd(54, 24)
```

With *infix functions*, however, the function name stands *between* the arguments. You've already seen this with the predefined `to` function, which creates `Pair`s (to put into a map):

```
"Dwayne" to 3.14159
```

Here, the function name `to` is surrounded by its two arguments, which brings us to the first limitation of infix functions: **an infix function must have exactly two parameters.** In the function call, the first argument is on the left

of the function name, and the second is on the right. Infix function calls don't need parentheses, which helps remove clutter from your code.

> Other infix functions you have already seen include `downTo` and `step`.

You can still call an infix function like a regular function as well. The following is equivalent to the example above:

```
to("Dwayne", 3.14159)
```

However, you can see that this significantly weakens code readability. Infix functions can improve readability for functions where it leads to a natural way of reading the function call.

Implementing an Infix Function

Kotlin has a function to repeat a string a given number of times:

```
val str = "Ho "
str.repeat(3)
```

Let's say you want to be able to call this using infix notation as `3 times "Ho "`. To do so, you can define an infix function called `times`:

```
infix fun Int.times(str: String) = str.repeat(this)
```

The `infix` modifier signifies that you're defining an infix function. Recall that an infix function must have two parameters, but the function signature in the example only has one. This is because an infix function must be either a class member or an extension function. The extended class becomes the first parameter type. Here, this is an extension function on the `Int` class, making the first parameter of type `Int`, and the second parameter is of type `String`. Inside the function body, you refer to the first parameter's value using `this`.

With this function definition, you can now use the syntax above:

```
infix fun Int.times(str: String) = str.repeat(this)


val greeting = 3 times "Ho "
println(greeting)
```

As you can tell, infix functions can sometimes help make the code read more naturally. This particular example can be further improved using *operator functions*, which will be covered in the next lesson.

Restrictions for Infix Functions

Infix functions must...

- be either *member functions* or *extension functions*.
- have exactly one additional parameter in the function signature (in addition to the extended class, which acts as the first parameter's type).
- *not* have varargs or default values.

Quiz

Infix Functions

1 Which of the following function calls uses infix notation?

Exercises

Implement an infix function `withoutLast` that removes the last appearance of a given substring in a given string.

Problem | Solution

```
// Add your code here
```

Call your infix function on several examples, including:

- `"What's up Kotlin?" withoutLast "up"` should return `"What's Kotlin?"`
- `"What's up up Kotlin?" withoutLast "up"` should return `"What's up Kotlin?"`
- `"What's up upKotlin?" withoutLast "up"` should return `"What's up Kotlin?"`

Hint: You can use Kotlin's `String.substringBeforeLast` and `String.substringAfterLast` in your function body.

Summary

Infix functions allow function calls where the function name stands between its two arguments.

- They can improve code readability (e.g., Kotlin's `to` and `step` function).
- They must fulfill some (weak) restrictions, which are easy to fulfill as long as the function logically has two parameters.
- You can define your own infix functions using the `infix` modifier in your function declaration.

The following lesson will take this concept a step further with so-called *operator functions*.

Operator Functions

Learn how to define your own operators, which limitations Kotlin poses on such operators, and how they lead to succinct code.

WE'LL COVER THE FOLLOWING

- Introducing Operators
  - Important Kotlin Operators
- Implementing an Operator Function
- Quiz
- Exercises
- Summary

Operator functions allow you to define the meaning of well-known symbols such as `+` and `-` for your own types.

Introducing Operators

Kotlin's operators allow calling functions using certain symbols. For instance, the infix `+` operator actually calls a `plus` function so that `a + b` translates to `a.plus(b)` under the hood.

Many operators use infix notation and therefore can be seen as an extension of infix functions. Instead of a function name, operator functions instead use an *operator symbol*. However, there are also operators using prefix and postfix notation:

- Prefix: `+a`, `-a`, `!a`
- Infix: `a * b`, `a..b`, `a == b`, ...
- Postfix: `a[i]`, `a()`

Kotlin has a predefined list of possible operators you can use (in contrast to languages like Scala or Haskell).

Important Kotlin Operators

Note: Some of the examples below require a custom operator definition to run. For instance, `"Coffee" * 3` doesn't work out of the box because Kotlin provides no such operator implementation.

Function Name	Operator	Example	Equivalent
Arithmetic Operators			
`plus`	`+`	`2 + 3`	`2.plus(3)`
`minus`	`-`	`"Kotlin" - "in"`	`"Kotlin".minus("in")`
`times`	`*`	`"Coffee" * 2`	`"Coffee".times(2)`
`div`	`/`	`7.0 / 2.0`	`(7.0).div(2.0)`
`rem`	`%`	`"android" % 'a'`	`"android".rem('a')`
Assignment Operators			
`plusAssign`	`+=`	`x += "Island"`	`x.plusAssign(“island”)`
`minusAssign`	`-=`	`x -= 1.06`	`x.minusAssign(1.06)`

timesAssign	*=	x *= 3	x.timesAssign(3)
divAssign	/=	x /= 2	x.divAssign(2)
remAssign	%=	x %= 10	x.remAssign(10)
Miscellaneous Operators			
inc	++	grade++	grade.inc()
dec	--	'c'--	'c'.dec()
contains	in	"A" in list	list.contains("A")
rangeTo	..	now..then	now.rangeTo(then)
get	[…]	skipList[4]	skipList.get(4)
set	[…] = …	array[5] = 42	array.set(5, 42)

For a full list of all operators, check out the Kotlin documentation.

Implementing an Operator Function

You can define the meaning of any of the allowed operators for any of your own types. **The declaration is similar as for infix functions, except that you use the `operator` modifier**:

```
operator fun Int.times(str: String) = str.repeat(this)
```

Remember that the function name can only be one of the predefined operators, whereas infix functions can have any name (recall your `withoutLast` function for instance). Also, not all operators have two parameters, and not all operators use infix notation.

With the function definition above, you can now repeat strings like this:

```
val greeting = 3 * "Ho "
```

This works because the `*` operator maps to the `times` operator function. Refer to the table above and the Kotlin documentation for all mappings.

Quiz

Operator Functions

1 Which of the following function names can you use to define an operator in Kotlin? You can select multiple answers.

COMPLETED 0%

1 of 3

Exercises

Implement an operator function that returns whether a `String` contains *all* `Char`s in a given list (`List<Char>`).

Problem | Solution

```
// Add your code here
```

You should be able to call the function as follows:

```
val requiredChars = listOf('x', 'y')
println(requiredChars in "xyzäöü")  // true
println(requiredChars in "x你z")  // false
println(requiredChars in "y好ä")  // false
```

Summary

Operators allow using predefined symbols to call a function.

- While this can improve code readability in certain cases, it should be used judiciously, since operators can quickly become confusing.
- There are dozens of allowed operators and many predefined operator implementations, such as `17++`, `"a" in "string"`, or “6 * 7”.
- To define a custom operator, you use the `operator` modifier and define either a member or extension function.

Congratulations! You now have a good understanding of the different types of functions in Kotlin and how to use them.

In the following section, you can recap the main points of each section in the course and find great resources to continue your Kotlin journey.

Recap

Recap what you learned about Kotlin, its principles, and how they drive language features.

WE'LL COVER THE FOLLOWING

- Variables and Mutable Types
- Nullable Types and Type Inference
- Conditions and Expressions
- Collections
- Loops
- Functions
- Summary

You now have a solid understanding of the basics of Kotlin. Before we explore ways to continue your Kotlin journey, let's briefly recap all that you learned in this course. After all, repetition is one of the keys to learning.

Variables and Mutable Types

- Kotlin favors the use of read-only variables.
  - Use `val` to create a read-only variable.
  - Use `var` to create a mutable variable.
  - First, note how declaring a read-only variable takes no more characters than a mutable one, in contrast to many other languages (e.g., Java requiring the `final` modifier).
  - Second, note that variable names in your code are usually nicely aligned because the two keywords have the same number of characters.
- In alignment with this, Kotlin also differentiates between read-only and mutable collection types, again favoring the use of read-only ones.
  - For instance, a simple `listOf(...)` will give you a read-only list.

- To get a mutable list, you have to explicitly write `mutableListOf(...)`, making read-only the default case.

Nullable Types and Type Inference

- One of Kotlin's main features regarding safety is nullability anchored in the type system.
  - Every type is non-nullable by default.
  - Thus, every variable is either explicitly nullable or cannot be `null` (to be precise, there are hacky ways to set `null`, but you won't usually come across these).
  - You should prefer non-nullable types over nullable ones because it simplifies code and prevents the common null pointer exception.
- The Kotlin compiler can infer the type of most expressions.
  - For instance, declaring the type of a variable is almost always optional.
  - You can always choose to declare types explicitly to improve readability or to program against a more abstract type.

Conditions and Expressions

- Kotlin has `if` and `when` to define conditions.
  - `when` can match against a variety of conditions and is therefore more powerful than `switch` from other languages.
- Both `if` and `when` are expressions in Kotlin.
  - Thus, you can assign a variable to the result of a condition.
    - The value is defined by the value in the last line of each condition block.
  - If a function contains only a condition, you can rewrite it using shorthand notation when using the condition as an expression.
- As a side note, most constructs in Kotlin are expressions because it allows for a more functional style of programming.

Collections

- The Kotlin Collections API directly builds upon the Java Collections API.
  - In contrast to Java, it clearly separates between read-only and

mutable collections.

- The common collection types are `List`, `Set`, and `Map`.
  - A **list** is an ordered linear data structure to store multiple elements and can contain duplicates.
  - A **set** is an unordered linear data structure to store multiple elements and cannot contain duplicates.
  - A **map** is an unordered nonlinear data structure to store multiple key-value pairs and can contain each key only once.

Loops

- Kotlin offers `for` and `while` loops.
  - `while` allows repeating a block of code until a given condition is no longer true.
  - `for` allows iterating over any `Iterable` object.
    - This includes collections, string, ranges (`1..10`), arrays, etc.
    - This differs from the `for` loop you may know from other languages where you define a loop counter variable.
      - It acts like a “for-each” loop, which is also available in many other languages.

Functions

- Kotlin offers several interesting features regarding function declarations.
  - For functions with a single expression in their body, you can use shorthand notation to avoid clutter.
  - Function parameters can have default values, eliminating the need for function overloading 90% of the time.
  - When calling a function, you can explicitly denote the parameter names for each argument to increase readability in certain cases.
  - Extension functions allow virtually extending the API of a type.
    - This is extremely useful to smoothen rough edges in third-party APIs.
  - You can define infix functions to write the function name between its two arguments, which can improve readability.
  - Lastly, you can implement a predefined list of operator functions on any type you want.

any type you want.

- Since it focuses on functional programming, Kotlin offers even more interesting features with regards to lambda functions, which were not covered in this course.

Summary

Congratulations! 🎉 Having finished this course, you're now able to write simple Kotlin scripts and applications. Also, with this solid understanding of the language basics, you'll be able to tackle more advanced material, such as functional and object-oriented programming (see the next lesson).

Thank you for making it through the entire course! I hope you enjoyed the journey and learned a lot along the way.

In the final lesson, I'll leave you with a hand-selected list of further resources about Kotlin that I'd recommend as the next step if you want to learn more about this fantastic programming language.

Next Steps

Discover the best resources to continue your Kotlin journey, and leave your feedback for the course!

WE'LL COVER THE FOLLOWING

- Free Resources
- Further Resources
  - Online Courses
  - Books
- Feedback?

Now you have all the knowledge required for intermediate and advanced Kotlin material.

This lesson offers useful resources to point you in the right direction.

> **Before you continue on your Kotlin journey, please leave feedback by clicking the "Send feedback" button at the bottom of the page to let me know:**
>
> - What did you like? What could be improved?
> - Would you like me to create a more advanced Kotlin course on Educative?
> - Which other topics do you want to learn about?
>
> **Any feedback is greatly appreciated. Thanks for finishing this course and I hope you enjoyed it!**

Free Resources

- **Official Kotlin language documentation**

✔ Well-written and provides up-to-date info directly from the makers of Kotlin.

✖ Not as engaging to work through, no exercises, not hands-on.

- **Official Kotlin Koans**

  ✔ Hands-on coding exercises, can be done directly in the browser or in your IDE, directly from Jetbrains

  ✖ You should already know the respective language features, not a tutorial

- Kotlin Slack

  ✔ Great place to ask all your questions and learn about all sides of Kotlin

  ✔ Brilliant community with many Kotlin developers ready to help

- Talking Kotlin podcast

  ✔ Directly from Jetbrains, diverse range of topics

  ✖ Often very specific and/or advanced topics

- Kotlin for Java Developers online course

  ✔ Directly from Jetbrains, goes beyond the basics

Further Resources

Online Courses

- **Kotlin for Android & Java developers: Clean Code on Android**

  ✔ Build your own Android app with Kotlin

  ✔ Recaps the basics and quickly moves on to intermediate and advanced language features

  - This is one of my Udemy courses

- **Kotlin for Beginners**

  ✔ Engaging videos, quizzes, and coding exercises

❌ The first half of the course is very similar to what you learned here, but it may be a good way to recap the topics

- This is one of my Udemy courses

Books

- **Kotlin for Android app development**

  ✔ Covers Kotlin basics, functional programming, object orientation, asynchronous programming, Android app development, best practices, and more

  - This book, which I wrote, covers much more than this introductory course

- **Kotlin in Action**

  ✔ Great resource for the Kotlin language itself, written by two members of the Kotlin team at Jetbrains

  ❌ Does not cover using Kotlin on Android or other specific platforms

---

Feedback?

```
if (youAlreadySentYourFeedback) {
  println("Thank you so much!")
} else {
  println("Please do leave your feedback using the 'Send feedback' button below.")
  println("I'd be thrilled to hear from you!")
}
```